

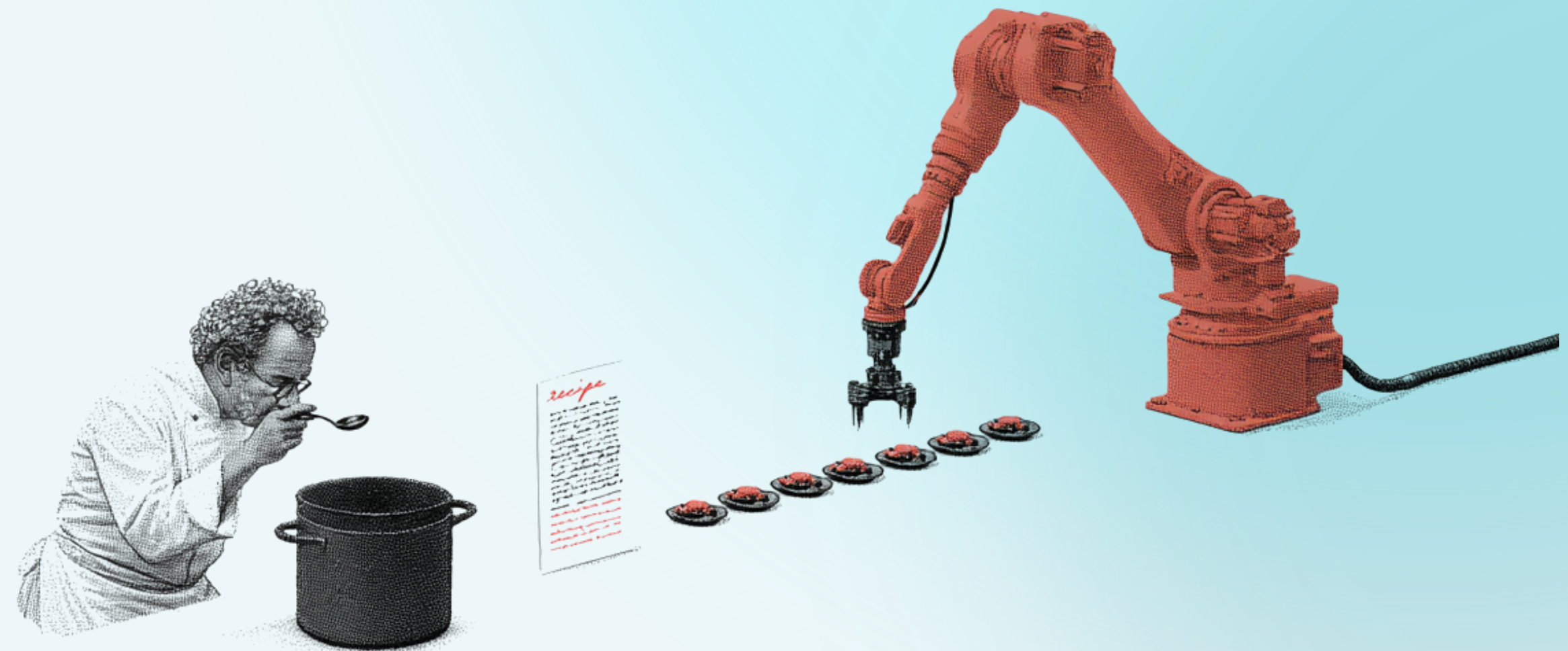
CASE STUDY · 01 / 03

exo-designer

design production for an engineering org

Design quality that keeps pace with how fast engineering ships.

At realfast, most user-facing interface code is now written by AI agents. I built exo-designer to give those agents the design system before they write a line, so every screen is on-brand and accessible the first time, with no review step and no designer added to the pod.



Manas Vaze · Staff designer, realfast

mnsvz23.github.io/website-2026

ABOUT REALFAST

Production software in weeks, with senior engineers and frontier models.

An AI-native software delivery firm: legacy modernisation, AI agent workflows, Salesforce implementations, and AI transformation engagements, on an in-house platform called Exo. Pods of two to five senior engineers, agents doing most of the building, outcomes tied to metrics agreed before work starts. Clients include Z47, AVPN, TASC, Houzbay, BoardRoom, and Hypernative.

~60%

gross margin, as reported

26×

revenue growth in 2026, as reported

BACKED BY



The founders set the bar. They built and sold one engineering firm, then ran product and technology at Gojek.



Sidu Ponnappa

CEO, Co-founder

Co-founded C42 Engineering, acquired by Gojek in 2015. Then CIO of Gojek Singapore, MD of Gojek India, and a Gojek Group board member.



Aakash Dharmadhikari

CPTO, Co-founder

Co-founded C42. Group Chief Product Officer and board member at Gojek. Later CPO at Jiva.



Steve Sule

CFD, Co-founder

Former Partner at Edith Grove Capital, previously Arma Partners and M&A at Mahindra & Mahindra. Joined as an advisor, became a co-founder.

Sources: realfast.ai/about · pmfevals.substack.com/p/the-neofirm-challenging-indias-it · yourstory.com/2024/10

Manas Vaze · Staff designer, realfast

mnsvz23.github.io/website-2026

One designer, many engineering pods, and every pod shipping a slightly different brand.

WHAT THE ORGANISATION GAINS

01

A brand that holds everywhere

Internal tools end up in client demos, partner screenshots, and hiring loops. When every screen shares one visual language, the promise of speed with quality is reinforced rather than undercut.

02

Accessibility without a checklist

Labels, named controls, and contrast are WCAG basics and real exposure on client deliverables. Built into the default, they stop being something anyone has to remember.

03

Review time back to the team

A designer reviewing a finished tool is redesigning it. Removing that step gives pods their Thursday back, and gives the designer time for work only a designer can do.

My job at realfast was to build design production infrastructure and pipelines for an engineering organisation, so that every screen the org ships comes out well designed, even though no designer touches most of them.

When I joined, every internal tool and client interface was built by whoever needed it, usually with an AI assistant writing the code. Each one worked fine on its own. Put them side by side and you could see the drift: different colours, different form controls, missing labels, spacing that changed from screen to screen. Nobody was being careless. There was simply no standard for the tools to follow.

LLMs can produce a settings page in under a minute. Without guidance, they produce the average of every settings page they have ever seen, which is nobody's brand. I wanted them to produce realfast's instead, at the same speed.

Instead of checking screens after they were built, I gave the AI the rules before it started.

BEFORE • REVIEW AFTER THE FACT

the loop that never ended

Request

AI writes UI

Designer reviews

Rework

Ship, eventually

My first attempt was a linter. It scanned, flagged, and fixed what it could, but it never changed anything upstream. Each audit found the same issues, we cleaned them up, and the next feature brought them back.

AFTER • RULES BEFORE THE CODE

CLAUDE.md is read at the start of every session

Rules in project

Request

AI writes governed UI

Self-check

Ship

I moved the design system into the file the model reads first: clear rules, examples of right and wrong, and a checklist it runs before it shows any code. The compliant version is now the first version. The audit stayed on as reassurance.

- THREE PRINCIPLES
- Live where the code is written

Show, don't tell

Right by default, checked on request

I directed the build, Claude Code did the typing, and I cut whatever did not earn its place.

22 DEC

Rules as a project file, not a Figma library
Engineers and agents never open Figma. The system had to live where code is written.

2 JAN

Built a UI compliance dashboard, deleted it three days later
Nobody opened it. A report that lives somewhere else is a report nobody reads.

2 JAN

Graded the audit on usability, not lint rules
Six weighted categories, feedback states counting most. A screen with no loading state is broken even when every colour is right.

5 JAN

One set of skills for React, Salesforce, and Rails
The tool detects the stack and adapts. One thing to learn, whatever the project.

I DECIDED	CLAUDE CODE PRODUCED
01 The thesis: rules before the code, not review after it	01 Scanners for accessibility, theming, and component usage
02 What counts as a violation, and how much each category weighs	02 Auto-fixers for colours, spacing, and labels
03 The role of every one of the sixteen brand colours	03 Seventeen pattern templates, to my specification
04 Killing the dashboard and staying in the terminal	04 The CLI, installer, and generator plumbing
05 The wording of every rule. Rules are copy.	05 First-draft documentation, which I rewrote

Eight skills, three jobs. Engineers keep working the way they always did, and the screens come out right.

- 1

Set up

ONCE PER PROJECT

Detects the framework, writes the theme and CSS variables, drops the design rules into the project.

/design-init /design-theme
- 2

Make

EVERY DAY

New and rebuilt screens come out compliant, built from 17 reference patterns.

/design-generate /design-redesign
- 3

Check

WHEN IT MATTERS

A safety net, not the mechanism. Quick pass, weighted audit, auto-fix, team report.

/design-check /design-audit /design-fix /design-report

```
claude • internal-tool

$ npm install exo-designer
  exo-designer: skills installed. Next: type /design-init

> /design-init
  Detected React + Tailwind. Applied preset: RealFast Branding.
  Wrote .design-agent/theme.json, src/styles/theme.css, CLAUDE.md

> Build a user management page with a table, search, and filters.
  Using patterns: page-layout, data-table, search-filter, empty-state
  Self-check components ✓ tokens ✓ labels ✓ focus ✓ loading ✓
                  empty ✓ contrast ✓ spacing ✓ errors ✓ responsive ✓
  Created src/pages/users.tsx

> /design-audit
  0 violations across 14 files. Score 100/100.
```

0

hardcoded colours, native inputs, or unlabelled fields allowed on any screen. By rule, not by review.

NOT MEASURED YET

Hours of review removed, and how many teams adopted it. These numbers describe the system, not its effect. Next is a before-and-after audit of tools already shipped, then widening the checks to error recovery, then starting client engagements with the client's own palette and the same guardrails.

What it delivers today, what I have not measured yet, and where it goes next.

- 6 quality checks on every screen
weighted toward what users notice first: feedback, forms, brand
- 17 reference patterns
for the screens engineers build most, right the first time
- 3 frameworks from one set of skills
React, Salesforce, and Rails, one thing to learn

